

PROJECT DOCUMENTATION

Title: AI Career Assistant

1. Introduction

This project presents an AI-powered career assistant designed to analyze user resumes and answer queries based strictly on the uploaded content. The system leverages **Retrieval-Augmented Generation (RAG)** to combine document retrieval with large language model capabilities.

The application is divided into two main components:

- A **FastAPI backend** that handles resume processing, embedding generation, storage, and query answering
- A **Streamlit frontend** that provides an interactive user interface

The system uses **LangChain**, **FAISS vector storage**, and **Groq-hosted LLaMA 3** to deliver accurate, context-aware responses.

2. Objectives

- To build an AI system capable of analyzing resumes
- To implement Retrieval-Augmented Generation for contextual Q&A
- To ensure responses are strictly grounded in user-provided data
- To design a scalable backend using FastAPI
- To provide an interactive frontend using Streamlit
- To support multiple users through user-specific data filtering

3. Technologies and Libraries Used

3.1 Technologies

- Python: Core development language
- FastAPI: Backend API framework
- Streamlit: Frontend interface
- Groq API: LLM inference (LLaMA 3.3 70B)
- FAISS: Vector database for semantic search

3.2 Libraries

- langchain_groq: LLM integration
- langchain_core.prompts: Prompt templates

- langchain_text_splitters: Text chunking
- langchain_community.vectorstores: FAISS integration
- langchain_community.embeddings: HuggingFace embeddings
- PyPDF2: PDF parsing
- python-docx: DOCX parsing
- python-dotenv: Environment configuration
- requests: Frontend-backend communication

4. System Architecture

4.1 Workflow

Resume Upload → Text Extraction → Text Splitting → Embedding Generation → FAISS Storage → Query Input → Similarity Search → Context Retrieval → Prompt Construction → LLM Response

4.2 Component Description

Frontend (Streamlit)

- Accepts user ID and resume upload
- Sends API requests to backend
- Displays chat-based interaction

Backend (FastAPI)

Handles all core logic including file processing, vector storage, and response generation.

Resume Processing

- Extracts text from PDF, DOCX, and TXT files
- Ensures only valid and non-empty content is processed

Text Splitting

- Uses RecursiveCharacterTextSplitter
- Splits text into chunks with overlap for better retrieval

Embedding Generation

- Uses HuggingFace model all-MiniLM-L6-v2
- Converts text chunks into vector representations

Vector Storage (FAISS)

- Stores embeddings locally
- Supports similarity search for retrieval

User-Based Filtering

- Each chunk is tagged with a user_id
- Retrieval is restricted to the specific user's data

Retriever

- Fetches top-k relevant chunks based on query

Prompt Engineering

- Enforces strict rules:
 - Only answer from resume
 - No assumptions
 - Return "Not found in resume" if missing

LLM (Groq - LLaMA 3.3)

- Generates final responses using retrieved context

5. Features

- Resume-based question answering
- Retrieval-Augmented Generation pipeline
- Multi-user support using user ID filtering
- Support for PDF, DOCX, and TXT formats
- FastAPI backend with REST endpoints
- Interactive Streamlit chat interface
- Local FAISS-based vector storage
- Strict prompt control for reliable outputs

6. API Endpoints

POST /upload

Uploads and processes a resume

Input:

- user_id (form data)
- file (resume file)

Output:

- Success or error message

POST /ask

Answers questions based on stored resume

Input (JSON):

```
{  
  "user_id": "string",  
  "question": "string"  
}
```

Output:

```
{  
  "answer": "string"  
}  
]
```

GET /

Health check endpoint

7. Code Implementation

```
import os  
  
from fastapi import FastAPI, UploadFile, File, Form, HTTPException  
from pydantic import BaseModel  
from fastapi.middleware.cors import CORSMiddleware  
  
from dotenv import load_dotenv  
  
from langchain_groq import ChatGroq  
  
from langchain_core.prompts import ChatPromptTemplate  
from langchain_text_splitters import RecursiveCharacterTextSplitter  
  
from langchain_community.vectorstores import FAISS  
from langchain_community.embeddings import HuggingFaceEmbeddings
```

```
from io import BytesIO

from docx import Document

import PyPDF2


# -----

# Load ENV

# -----

load_dotenv()


GROQ_API_KEY = os.getenv("GROQ_API_KEY")


if not GROQ_API_KEY:

    raise ValueError("❌ GROQ_API_KEY not found in .env")


# -----

# FastAPI App

# -----

app = FastAPI(title="ResumeIQ AI")


app.add_middleware(

    CORSMiddleware,

    allow_origins=["*"],

    allow_credentials=True,

    allow_methods=["*"],

    allow_headers=["*"],

)


# -----

# LLaMA 3 via Groq

# -----
```

```

llm = ChatGroq(
    model="llama-3.3-70b-versatile",
    temperature=0,
    api_key=GROQ_API_KEY
)

# -----
# Embeddings (FREE)
# -----

embeddings = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2"
)

DB_PATH = "faiss_store"

# -----
# Vector Store
# -----

def load_vectorstore():
    if os.path.exists(DB_PATH):
        return FAISS.load_local(
            DB_PATH,
            embeddings,
            allow_dangerous_deserialization=True
        )
    return None

def save_vectorstore(vs):
    vs.save_local(DB_PATH)

def store_resume(user_id: str, text: str):

```

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=100  
)
```

```
chunks = splitter.split_text(text)  
metadatas = [{"user_id": user_id} for _ in chunks]
```

```
vs = load_vectorstore()
```

```
if vs is None:
```

```
    vs = FAISS.from_texts(chunks, embeddings, metadatas=metadatas)
```

```
else:
```

```
    vs.add_texts(chunks, metadatas=metadatas)
```

```
save_vectorstore(vs)
```

```
def retrieve_context(user_id: str, query: str, k: int = 4):
```

```
    vs = load_vectorstore()
```

```
    if not vs:
```

```
        return ""
```

```
    docs = vs.similarity_search(query, k=k)
```

```
    filtered = [  
        d.page_content for d in docs  
        if d.metadata.get("user_id") == user_id  
    ]
```

```
    return "\n".join(filtered)
```

```
def generate_answer(user_id: str, question: str):  
    context = retrieve_context(user_id, question)  
  
    if not context:  
        return " ⚠ Please upload a resume first."  
  
    prompt = ChatPromptTemplate.from_template("""  
You are a strict AI Career Coach.  
  
Rules:  
- Answer ONLY using the resume  
- If information is not present → say "Not found in resume"  
- Do NOT assume anything  
- Keep answers clear and professional  
  
Resume:  
{context}  
  
Question:  
{question}  
""")  
  
    chain = prompt | llm  
  
    response = chain.invoke({  
        "context": context,  
        "question": question  
    })  
  
    return response.content
```



```

# -----
# File Parsing
# -----

def extract_text(file: UploadFile) -> str:
    filename = file.filename.lower()
    content = file.file.read()

    if filename.endswith(".txt"):
        return content.decode("utf-8")

    elif filename.endswith(".docx"):
        doc = Document(BytesIO(content))
        return "\n".join([p.text for p in doc.paragraphs])

    elif filename.endswith(".pdf"):
        reader = PyPDF2.PdfReader(BytesIO(content))
        text = ""
        for page in reader.pages:
            page_text = page.extract_text()
            if page_text:
                text += page_text + "\n"
        return text.strip()

    else:
        raise HTTPException(
            status_code=400,
            detail="Unsupported file type (.pdf, .docx, .txt only)"
        )

# -----
# API Models

```

```

# -----

class Question(BaseModel):
    user_id: str
    question: str

# -----

# Routes
# -----

@app.post("/upload")
async def upload_resume(
    user_id: str = Form(...),
    file: UploadFile = File(...)
):
    try:
        text = extract_text(file)

        if not text.strip():
            raise HTTPException(400, "Empty or unreadable file")

        store_resume(user_id, text)

        return {
            "status": "success",
            "message": "Resume uploaded successfully"
        }

    except Exception as e:
        raise HTTPException(500, str(e))

@app.post("/ask")
async def ask_question(request: Question):

```

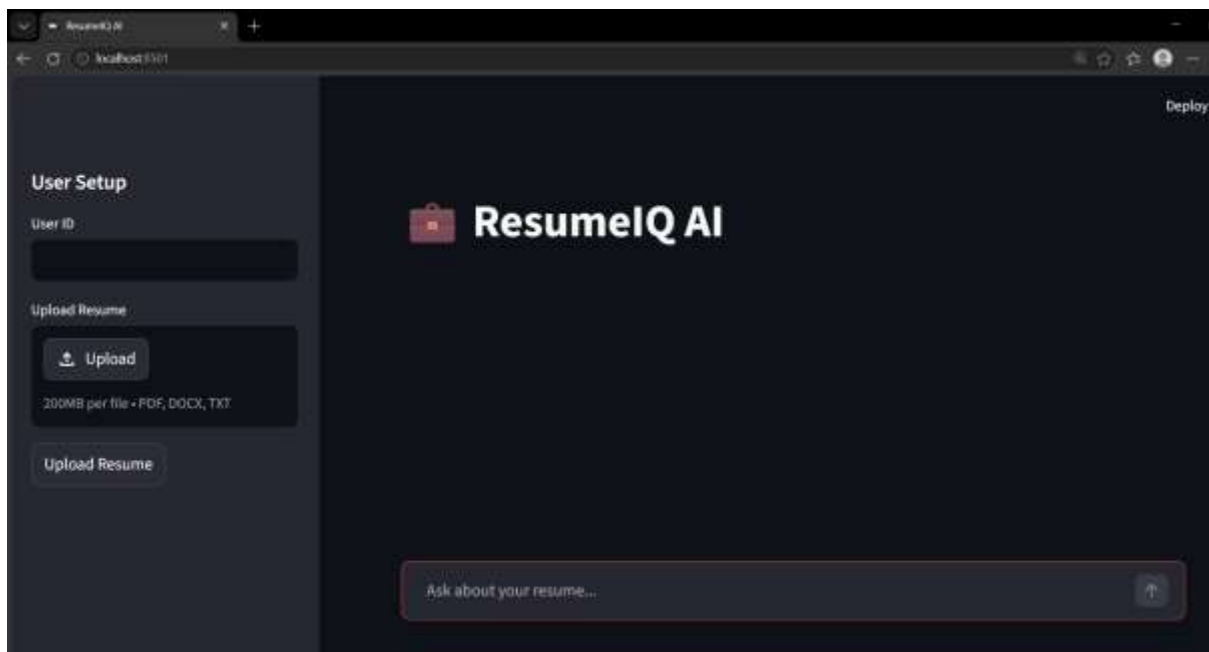
```
try:
    answer = generate_answer(
        request.user_id,
        request.question
    )

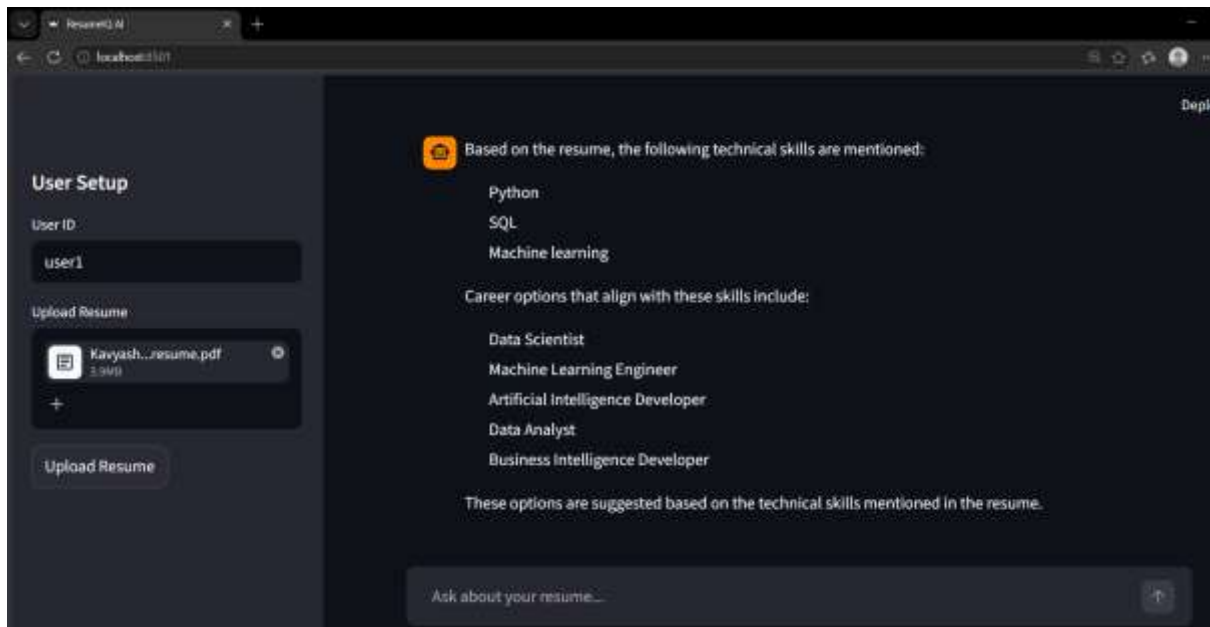
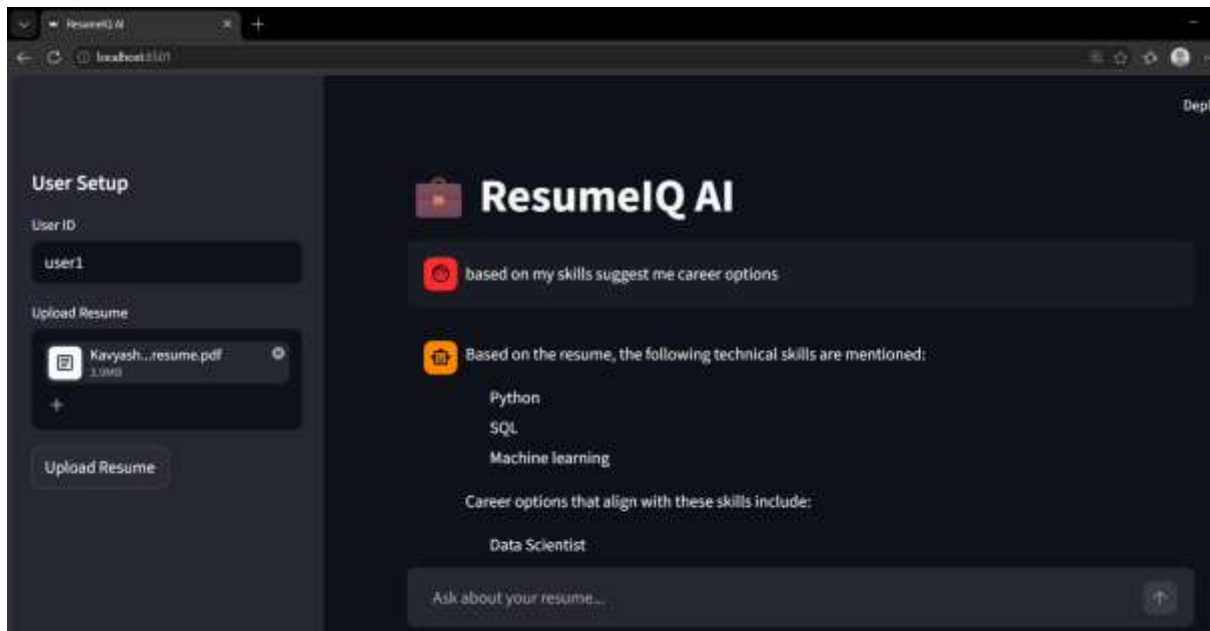
    return {"answer": answer}

except Exception as e:
    raise HTTPException(500, str(e))

@app.get("/")
def root():
    return {"message": "ResumeIQ AI running 🚀"}
```

8. Output





9. Conclusion

This project demonstrates the practical implementation of a Retrieval-Augmented Generation system for resume analysis. By combining vector search with large language models, the system ensures accurate and context-bound responses. The modular architecture enables scalability and extension into advanced career assistance platforms.